Installing Lazydocker

Before installing Lazydocker, make sure you have Docker installed (version >= 29.0). To verify if Docker exists, run:

```
$ docker -version
```

And to check the details, run:

```
$ sudo docker info
```

Now install Lazydocker in the Linux system by using the official installation script:

```
$ curl https://raw.githubusercontent.com/jesseduffield/lazydocker/master/scripts/install_update_linux.sh | bash
```

The binary is installed to *~/.local/bin* by default. If it's not in your *PATH*, make sure to add it too.

Before running any commands related to Docker, ensure the user has access. To do that, use the following command:

```
$ sudo usermod -aG docker $USER
```

Once Lazydocker is installed, start the Docker daemon using:

```
$ sudo docker ps
```

To start the application, use:

```
$ lazydocker
```

Monitoring running containers

To monitor a running container, first run a container:

```
$ docker run -d --name my-nginx -p 8080:80 nginx
```

Here, we ran a nginx container. Now, when we go to the Lazydocker TUI, we can find the details of the container (my-nginx) as seen in Figure 1. In the interface, you can click on the various options available to get detailed information without giving a host of Docker commands, which is the primary advantage of this tool.

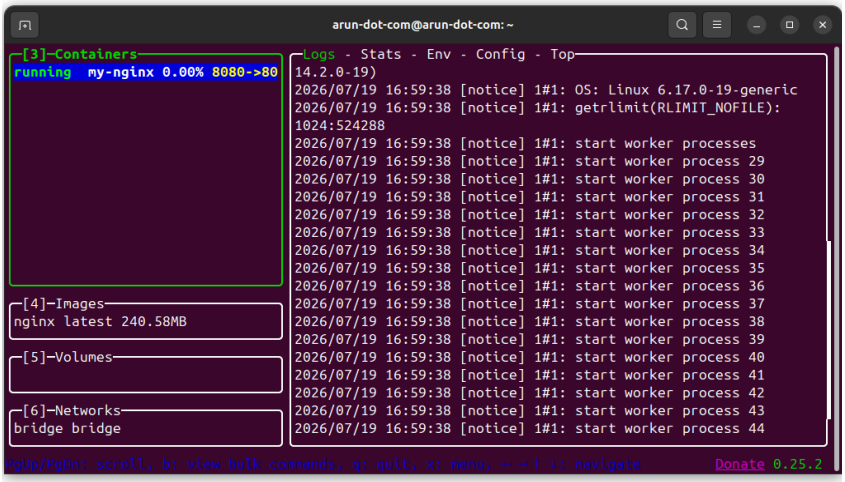

Figure 1: Lazydocker interface (TUI)

You can now:

- Select *my-nginx* with the arrow keys.
- Press *Tab* to switch between views.
- Open *Logs* to see container logs.
- Open *Stats* to watch CPU and memory usage.
- Open *Inspect* to view configuration details.

You can also visit *http://localhost:8080* in your browser to generate some activity.

Then, come back to the Lazydocker interface to see the relevant logs generated.

Lazydocker is especially useful when you're developing applications with multiple containers. For example, if you start a project with Docker Compose:

```
$ docker compose up -d
```

...and it creates frontend, backend, postgres and redis services, Lazydocker lets you:

- Monitor all services in one interface.
- Tail logs for any container.
- Check CPU and memory usage.
- Restart or stop services.
- Open a shell inside a container.
- Inspect networks, volumes, and images.

This makes it much easier to manage a multi-container application than repeatedly running *docker ps*, *docker logs*, and *docker stats* in separate terminals.

Executing Lazydocker in a container

Start Lazydocker and select the container with the arrow keys. Press *E* (or navigate to the Exec panel, depending on your Lazydocker version). Lazydocker will open an interactive shell inside the container. If the image doesn't include *bash*, it will typically fall back to *sh*.

Any required command can be executed inside the container. For example, I have executed a simple *echo* command as shown in Figure 2. Since this is a nginx container, you can even execute *$ curl http://localhost* to check whether the application inside the container is responding.

Cleaning up unused resources

Lazydocker makes it easy to remove unused Docker resources without remembering all the *docker prune* commands.

Start Lazydocker. Press *x* (in recent versions, this opens the *Cleanup* menu). You'll see options to remove unused resources, such as stopped containers, dangling images, unused images, unused volumes, unused networks, and